



CSE357: COMBINATORIAL STUDIES



Unit IV: Fundamentals of Programming Languages

- C/C++/Java Interview Questions Overview,,
- Variables, Data Types, and Memory,, Pointers and Storage, Classes,, Introduction to Object-Oriented
- Programming (OOP),, Compiler and Interpreter Basics,, Understanding Process Loading and Linking,,
- Techniques of Parameter Passing and Binding,, Storage Organization and Storage Classes,, Advanced
- OOP Concepts and Data Structures, Abstract Data Types (ADTs),, Data Structure Principles,,
- Advanced Programming Techniques,, Memory Handling in OOP Languages

Introduction to OOPs

Object-Oriented Programming is a programming paradigm where you can use a real world entity which is called an **Object**.

Let us consider an example

- ❑ **Attribute:** Name, Age, Color
- ❑ **Behaviour:** Singing, Dancing



Parrot



Object-Oriented Programming (OOP) is a programming approach where software is designed using **objects and classes**.

Instead of writing only functions and procedures, we model **real-world entities** such as:

- Person
- Car
- Student
- Bank Account

Each entity is represented as an **object**.

Object-Oriented Programming (OOP) Concepts

- **Key Concepts of OOP:**
- Encapsulation:
- Inheritance:
- Polymorphism:
- Abstraction:

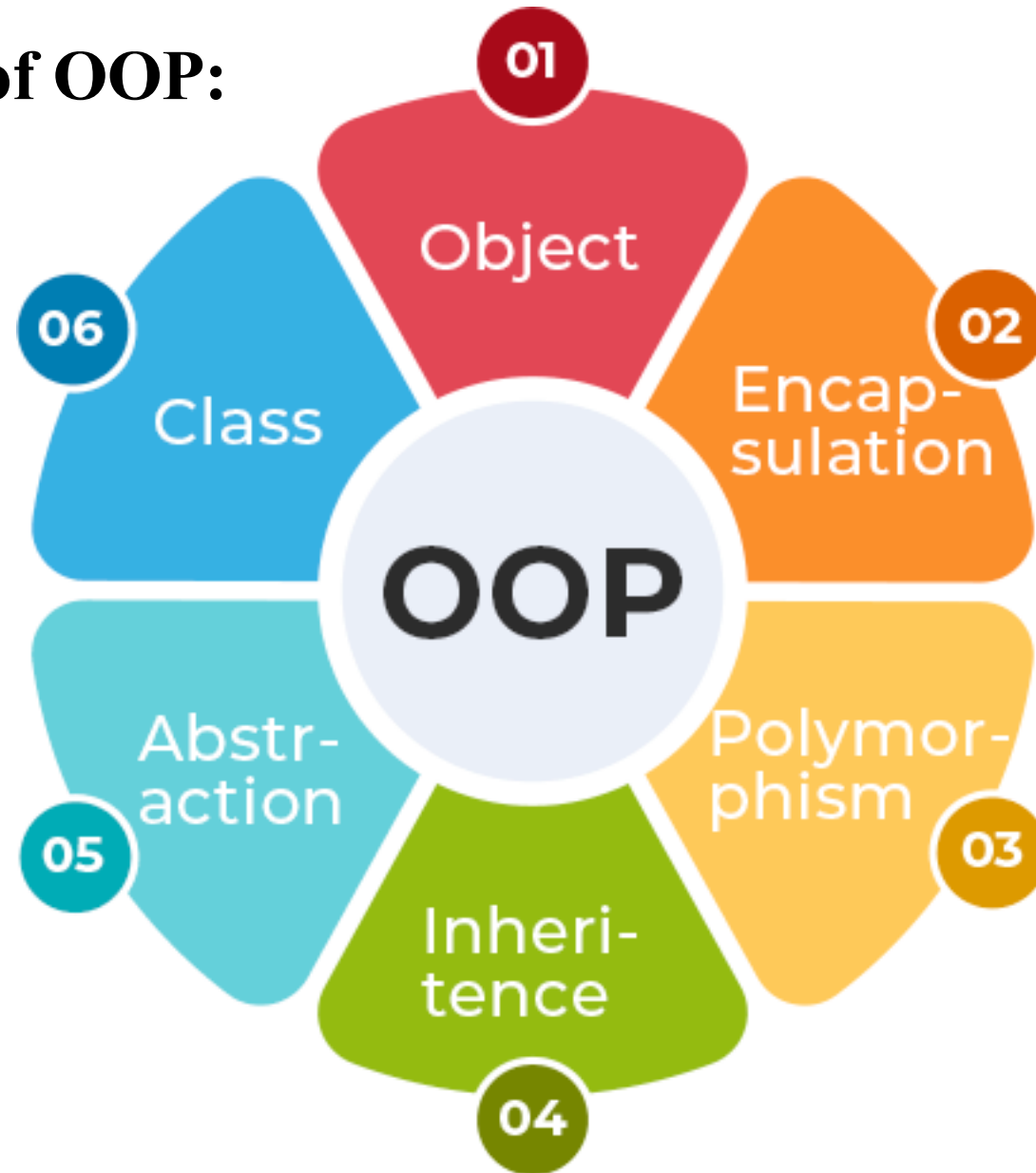




Benefits of OOP:

- **Modularity:** OOP promotes modular design, making it easier to understand, maintain, and modify software components.
- **Reusability:** OOP facilitates code reuse through inheritance and composition, leading to more efficient development and reduced duplication of code.
- **Flexibility:** OOP provides flexibility in designing and implementing software solutions, allowing for easier adaptation to changing requirements.
- **Scalability:** OOP supports scalability by enabling the creation of complex systems from smaller, reusable components.

Key Concepts of OOP:



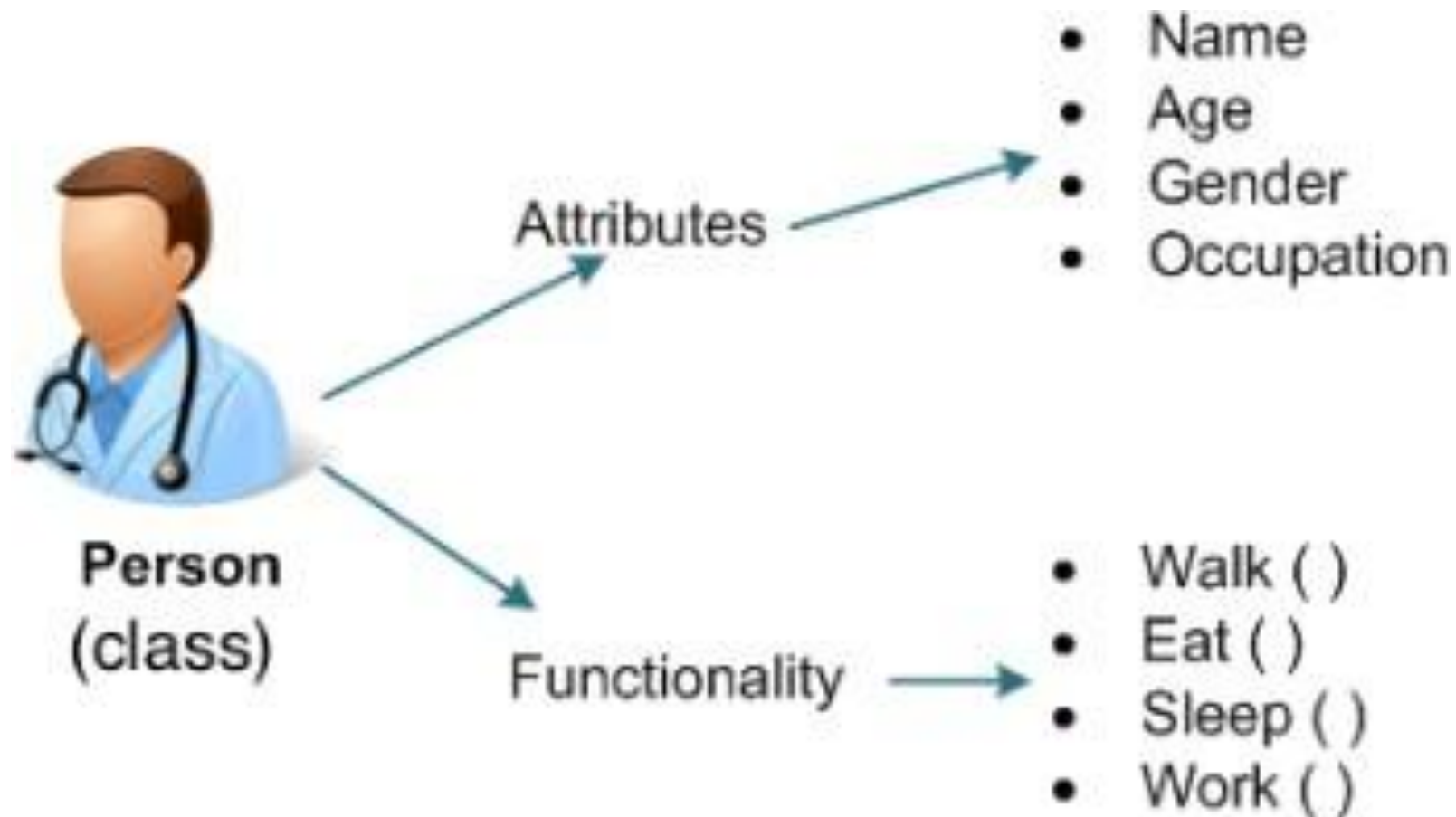
Classes and Objects

- A **class** is a **user-defined blueprint or prototype**
- Used to **create objects**
- Represents **common properties and methods** of objects
- Allows creation of **multiple objects with same behavior**
- Helps avoid **repetitive code writing**
- Improves **code reusability and organization**

A **Class** is a **blueprint or template** used to create objects.

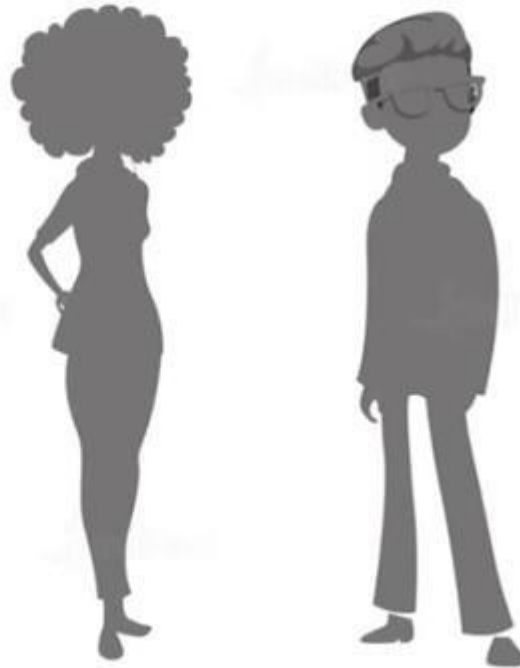
It defines:

- **Attributes (variables)** → properties of the object
- **Methods (functions)** → behavior of the object



Real-world OOP example

Considering Human Being is a **class**



Real-world OOP example

Name and Age are attributes (data members) of class MALE.



Class: MALE
Name: Victor
Age: 24

- Objects have a physical existence
- Class is just a logical definition

Real-world OOP example

Common body features and functions are **Class Attributes**

Every Human has:



NOSE



HAND



LEGS



HEART



EYES

Common Body
Parts:

Every Human has:



WALK



LISTEN



SPEAK



SEE



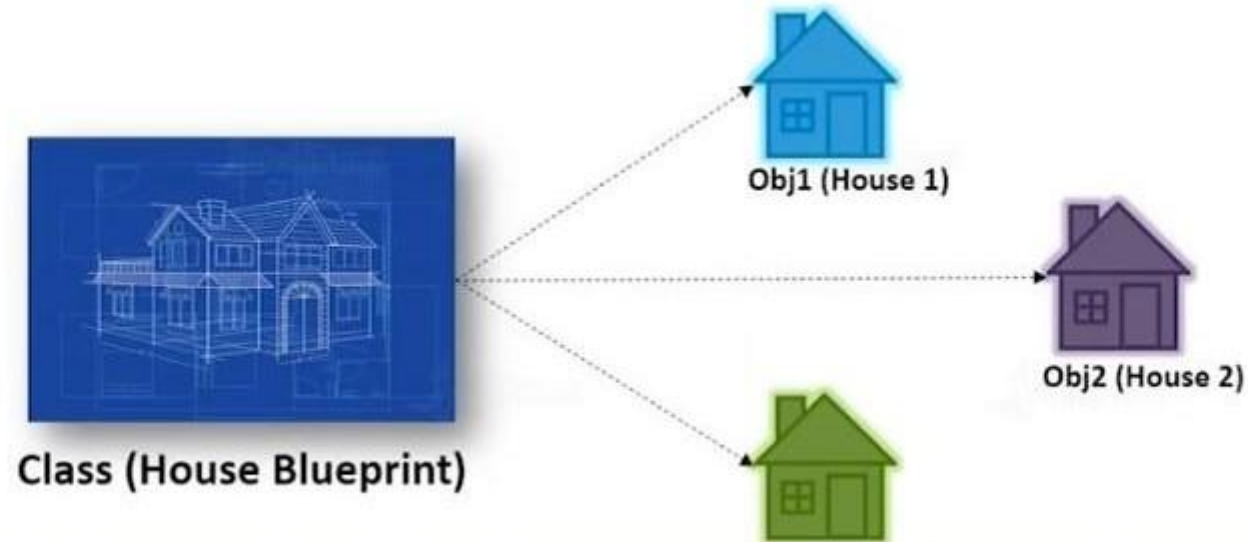
SMELL

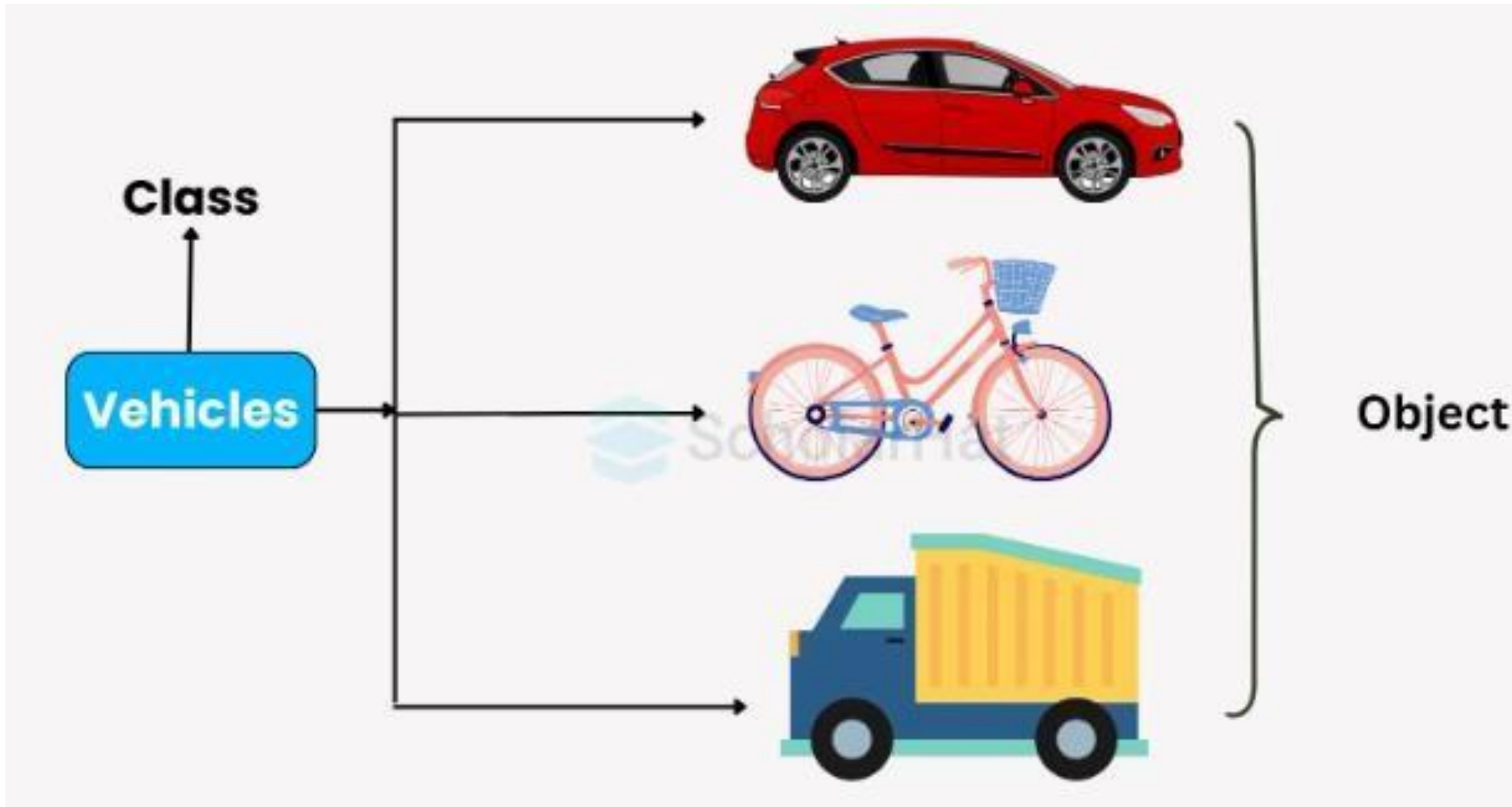
Common
Body
Function:

OOPs – Classes and Objects

What are Objects and Classes?

Class is a blueprint for an object and the objects are defined and created from classes(blueprint)





Access Specifiers: Access Specifiers define who can access the variables and methods of a class. They help in data protection and encapsulation.

Specifier	Symbol	Accessibility
Public	public	Accessible everywhere
Protected	protected	Accessible within class & derived class
Private	private	Accessible only inside the class

Access Type	Real-Life Example
Public	Notice board
Protected	University rules
Private	UMS login or password



Key Concepts of OOP

Encapsulation

- Bundling **data and methods** into a single unit (object)
- Hides **internal data** of the object
- Exposes only **necessary operations**
- Improves **security and data protection**

Encapsulation = Data Hiding + Controlled Access

Memory Trick

E = Egg Concept 

- Shell = Protection
- Inside = Data

 Encapsulation = Data inside protective shell

Encapsulation

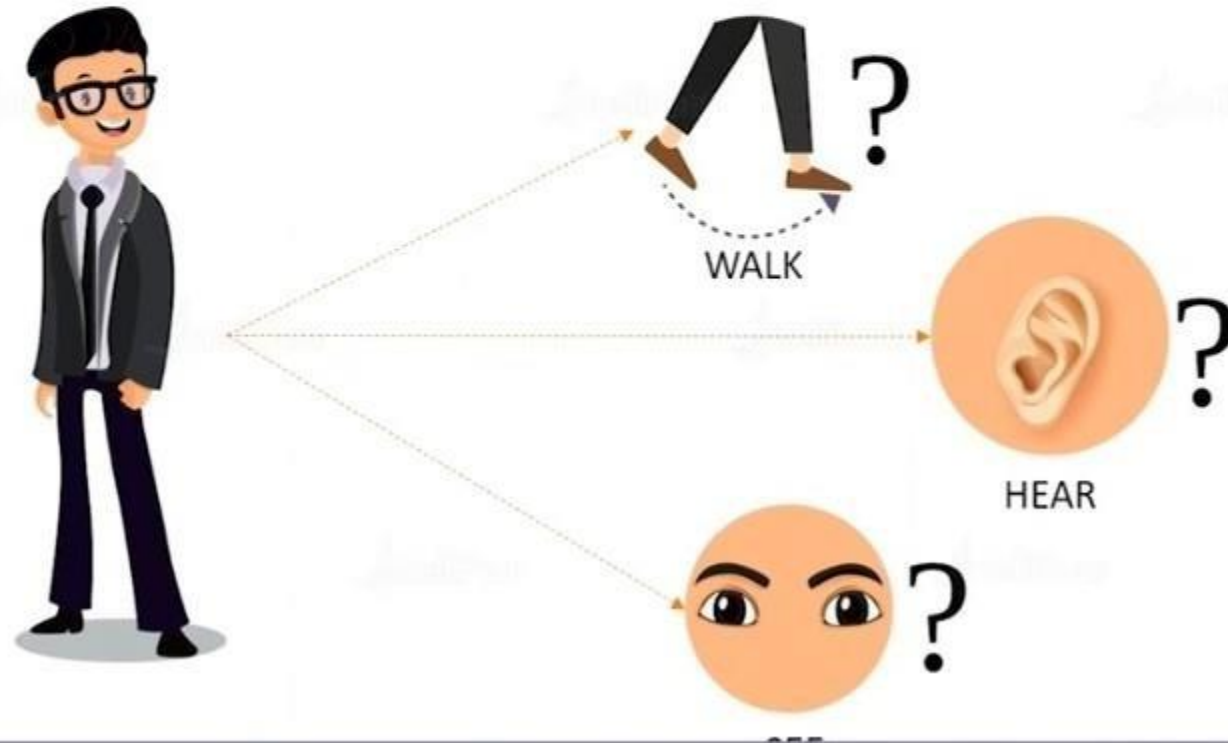
Encapsulation = Abstraction + Data Hiding. Abstraction is showing essential features and hiding non-essential features to the user.



While writing a mail you don't know how things are actually happening in the backend

Real-world OOP example

You don't know the detail of how you walk, listen or see.
i.e. its **hidden** or **Encapsulated**





ATM Component	OOP Concept
ATM Machine	Class
Account balance	Private Data
Withdraw / Deposit	Methods
User	Object



Why do we use Encapsulation?

- Protects data from **unauthorized access**
- Improves **security**
- Makes code **safe and maintainable**
- Controls **how data is accessed and modified**



Bank ATM

- You **cannot directly access bank balance**
- You must use **ATM methods**:
 - check Balance()
 - withdraw()
 - deposit()

☞ Data (balance) is **hidden**, access is **controlled** → Encapsulation



Key Concepts of Encapsulation

1. Data Hiding

- Hides the **internal state of an object**
- Prevents **direct modification of data**
- Ensures **data integrity and security**
- Allows **controlled access**

2. Access Control

Controls access using **visibility levels**:

- **Public**
- **Private**
- **Protected**
- **Helps manage object behavior**
- **Prevents unintended modifications**



3. Information Hiding

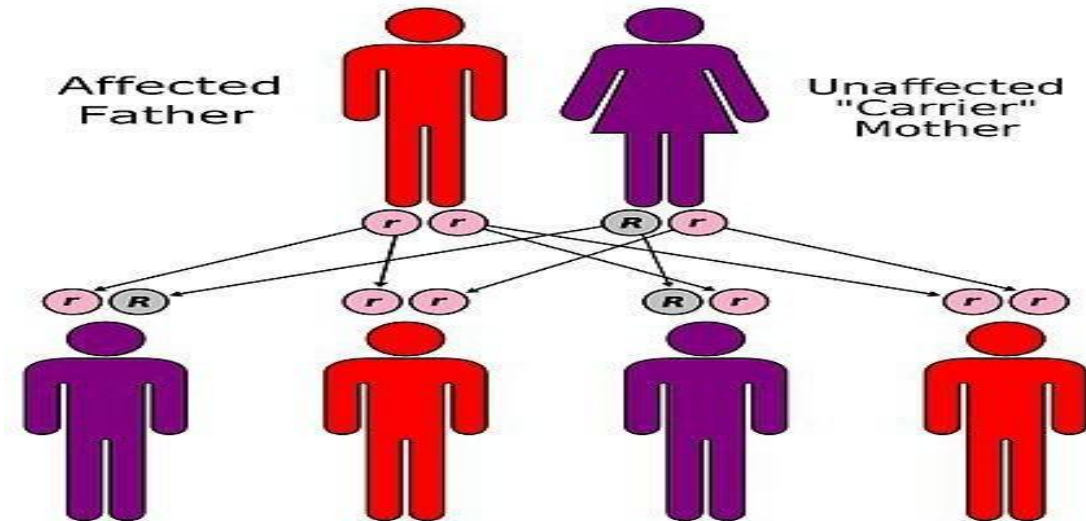
- **Exposes only essential details(ATM)**
- **Hides internal complexity(CAR)**
- **Improves code maintainability(Gpay)**

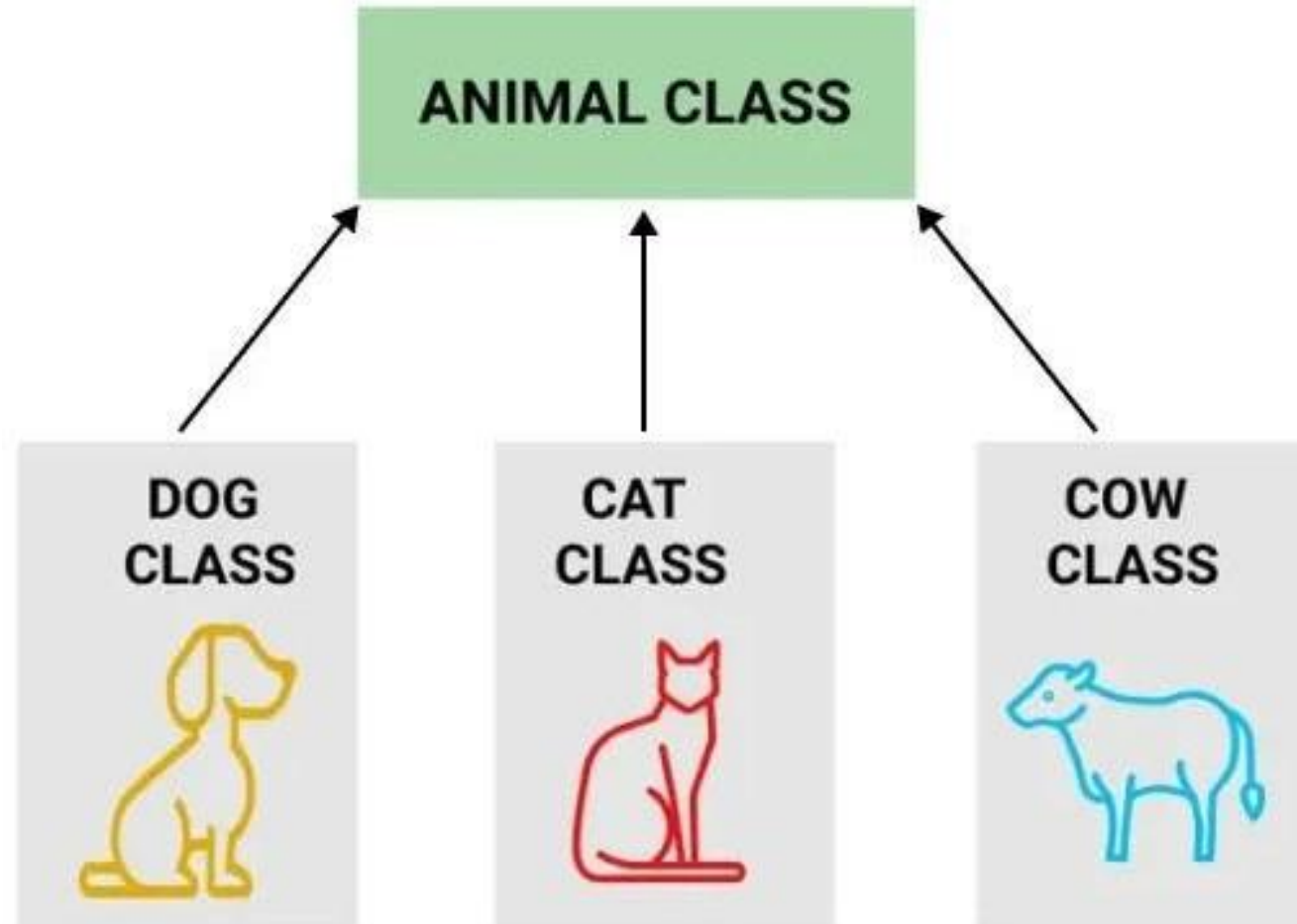
Inheritance and Polymorphism

Inheritance is a fundamental concept in Object-Oriented Programming (OOP) that allows a class (subclass or

Key Concepts of Inheritance:

- Superclass and Subclass:
- Inheritance Hierarchy:
- Code Reuse:
- Method Overriding:





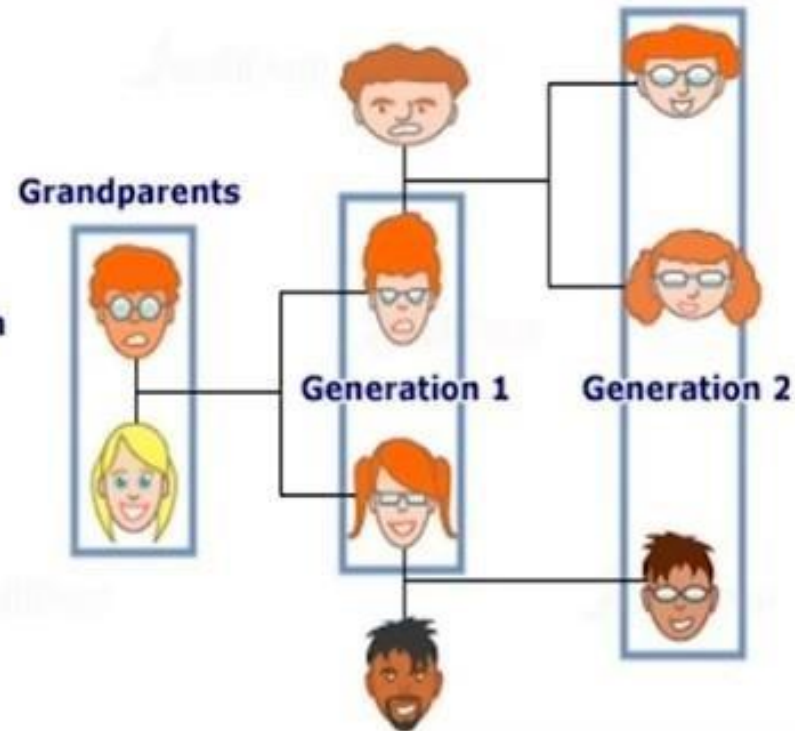


Key Concepts of Inheritance:

- **Superclass and Subclass:** The superclass is the class from which properties and methods are inherited, while the subclass is the class that inherits those properties and methods.
- **Inheritance Hierarchy:** Inheritance can form a hierarchy of classes, with each subclass inheriting from its superclass and potentially adding additional attributes or methods.
- **Code Reuse:** Inheritance enables the reuse of code by allowing subclasses to inherit and extend the functionality of their superclass without duplicating code.

One class acquiring the property of another class. For example, you would have inherited few qualities from your parents.

In a family tree, traits such as hair color and poor eyesight are passed from generation to generation.





Single Inheritance

Multiple Inheritance

Multilevel Inheritance

Hierarchical Inheritance

Hybrid Inheritance

Single Inheritance

Multiple Inheritance

Multilevel Inheritance

Hierarchical
Inheritance

Hybrid Inheritance

Single class inherits from a class



Parent Class

Child Class

1. Single Inheritance (One parent → One child)

Definition: A class inherits from only one base class.

Examples:

1. Person → Student

A student inherits basic details (name, age) from a person.

2. Vehicle → Car

A car inherits properties like speed and fuel type from a vehicle.

3. Animal → Dog

A dog inherits characteristics like eating and breathing from an animal.

Single Inheritance

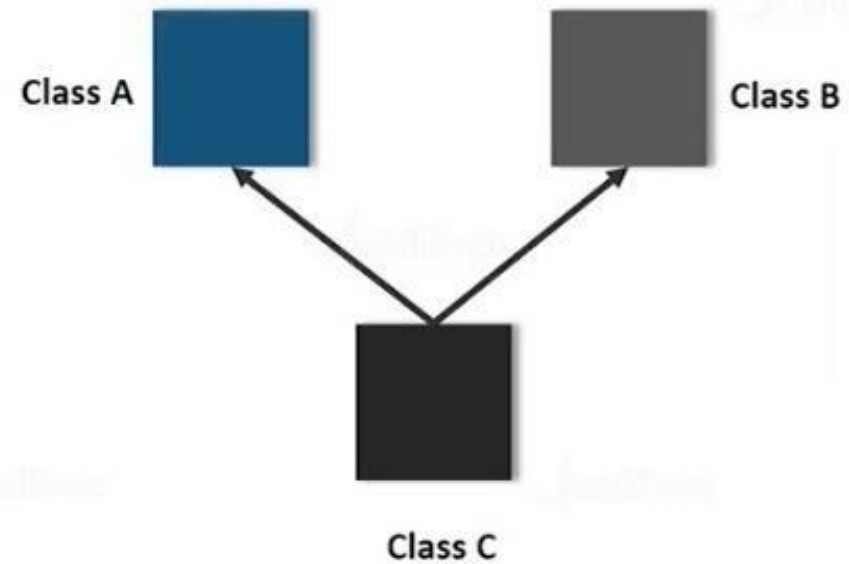
Multiple Inheritance

Multilevel Inheritance

Hierarchical
Inheritance

Hybrid Inheritance

A class inherits from multiple classes



2. Multiple Inheritance (Multiple parents → One child)

Definition: A class inherits from more than one base class.

Examples:

1. Teacher + Researcher → Professor

A professor has qualities of both teaching and research.

2. Printer + Scanner → All-in-One Printer

A machine performs both printing and scanning functions.

3. Musician + Actor → Entertainer

A person skilled in both music and acting.

Single Inheritance

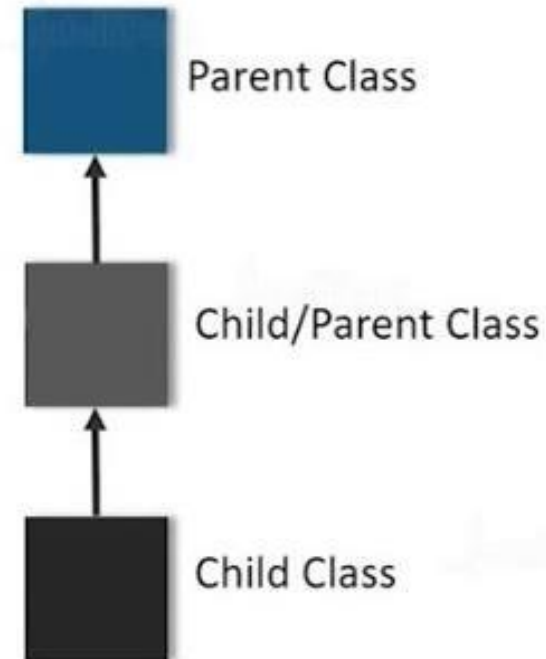
Multiple Inheritance

Multilevel Inheritance

Hierarchical Inheritance

Hybrid Inheritance

One class inherits from a class, which will inherit from another class



3. Multilevel Inheritance (Grandparent → Parent → Child)

Definition: A class inherits from a derived class, forming a chain.

Examples:

1. Living Being → Animal → Dog

Dog inherits from Animal, which inherits from Living Being.

2. Vehicle → Car → Electric Car

Electric car inherits from car, which inherits from vehicle.

3. Person → Employee → Manager

Manager inherits from employee, which inherits from person.

Single Inheritance

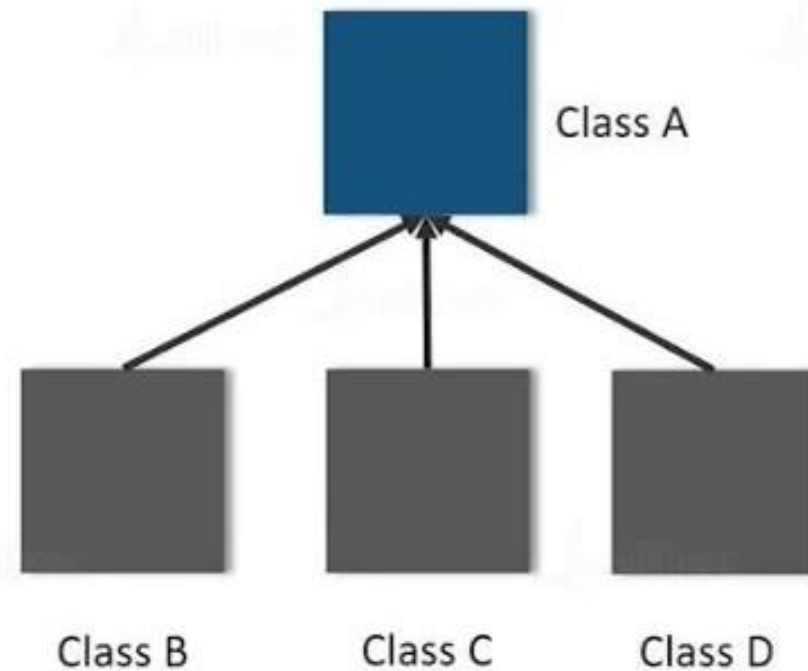
Multiple Inheritance

Multilevel Inheritance

Hierarchical
Inheritance

Hybrid Inheritance

More than one class inherits from a class



4. Hierarchical Inheritance (One parent → Multiple children)

Definition: Multiple classes inherit from a single base class.

Examples:

1. **Vehicle → Car, Bike, Truck**
All inherit common vehicle properties.
2. **Shape → Circle, Rectangle, Triangle**
All inherit properties like area/perimeter.
3. **Employee → Manager, Developer, Tester**
All inherit basic employee details.

Single Inheritance

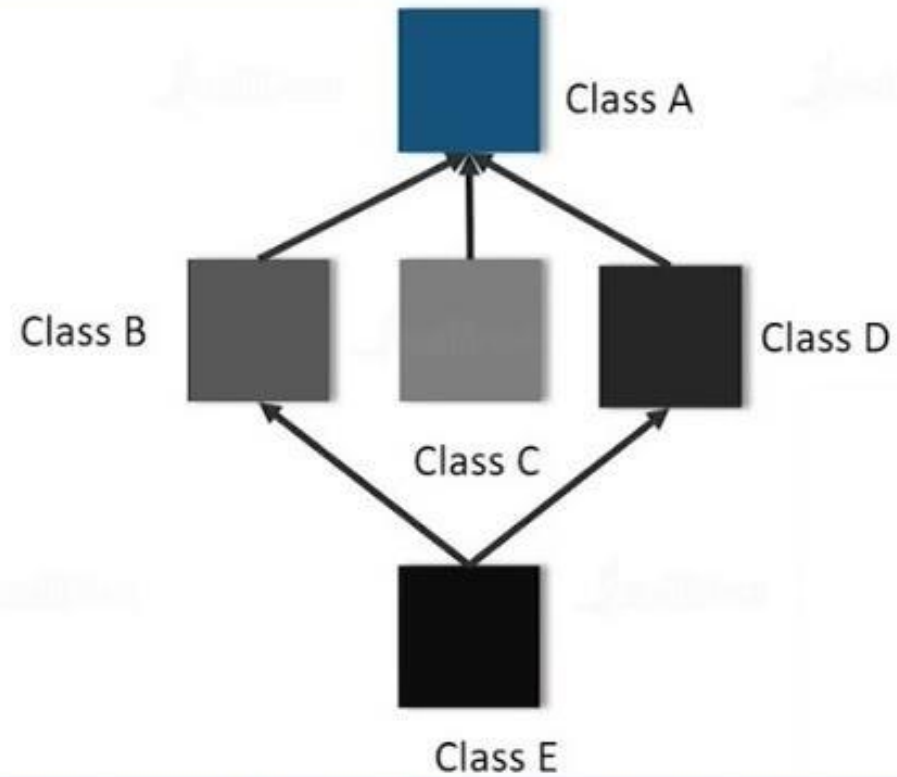
Multiple Inheritance

Multilevel Inheritance

Hierarchical
Inheritance

Hybrid Inheritance

Combination of any two kinds of inheritance



5. Hybrid Inheritance (Combination of types)

Definition: Combination of two or more inheritance types (e.g., multiple + hierarchical).

Examples:

1. **Person → (Teacher, Researcher) → Professor**
Combines hierarchical and multiple inheritance.
2. **Device → (Printer, Scanner) → Multi-function Device**
Combination of hierarchical + multiple.
3. **Vehicle → Car → Electric Car + Autonomous System → Self-driving Car**
Combines multilevel and multiple inheritance.

The `super` keyword in Java is used to refer to the immediate parent (superclass) object. It helps access parent class members when working with inheritance.

◆ What is `super`?

`super` is a reference variable that points to the parent class object.

👉 Used when child and parent have **same properties or methods**.

Access Parent Class Variable

◆ Real-Life Example: Employee Salary System

A company has:

- Base salary (parent class)
- Bonus salary (child class also defines salary)

If both have a variable named `salary`, we use `super` to access the **parent salary**.

👉 Idea: Distinguish between parent and child data.

◆ Example: `==` vs `equals()` in Java

`</>` Java

```
String s1 = new String("Hello");  
String s2 = new String("Hello");  
  
System.out.println(s1 == s2);      // false  
System.out.println(s1.equals(s2)); // true
```

◆ Explanation:

- `s1 == s2` → false
👉 Because both refer to different objects in memory
- `s1.equals(s2)` → true
👉 Because both have the same content ("Hello")

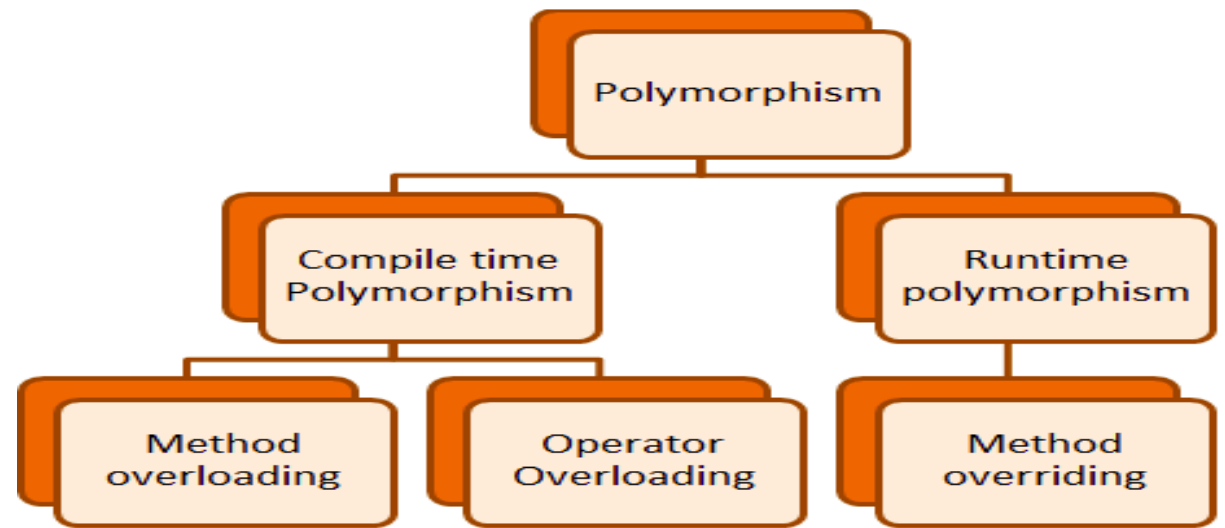


What is the purpose of the 'this' keyword in Java

The this keyword in Java is used to **refer to the current object of a class**, allowing access to its variables, methods, and constructors.

Polymorphism

- Allows **different objects to be treated as one common type**
- Enables **single interface, multiple implementations**
- Achieved through:
- **Method Overloading**
- **Method Overriding**
- Improves **flexibility and scalability**



Real-world OOP example

'She' can be a woman, wife, mother and a teacher at the same time which is many forms or **Polymorphism**



WOMAN



WIFE or
MOTHER



TEACHER

A person acts as an employee at work, a father at home, and a customer in shopping malls.



Operator Example +

- $5 + 3 = 8$ (addition)

- "Hello" + "World" = "HelloWorld" (concatenation)

☞ Same symbol +, different operations.

◆ Compile-Time Polymorphism (Method Overloading)

👉 **Concept:** Same action, different ways to perform it (decided before execution)

✅ Example 1: Calculator

- A calculator can:
 - Add 2 numbers
 - Add 3 numbers
- Same operation → `add()`, but different inputs

👉 **Real-life mapping:**

- `add(int a, int b)`
- `add(int a, int b, int c)`

◆ Run-Time Polymorphism (Method Overriding)

👉 **Concept:** Same action, but different behavior depending on object (decided at runtime)

✅ Example 1: Employee Salary

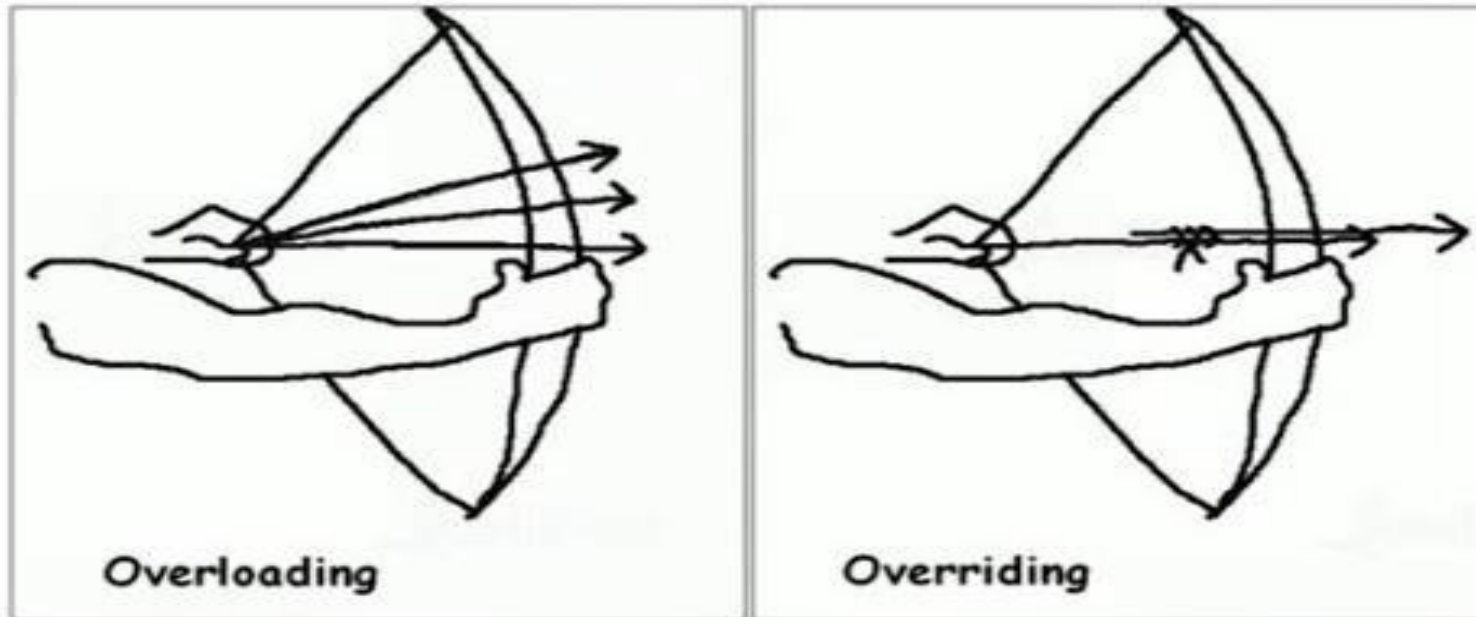
- Same method → `calculateSalary()`
- Behavior differs:
 - Full-time employee → Fixed salary
 - Part-time employee → Hourly basis
 - Intern → Stipend

Key Concepts of Polymorphism:

- **Method Overloading:** Polymorphism can be achieved through method overloading, where multiple methods with the same name but different parameter lists are defined within a class.
- **Method Overriding:** Polymorphism can also be achieved through method overriding, where a subclass provides its own implementation of a method inherited from its superclass

Overriding vs Overloading

Developers sometimes get confused between them





Method Overloading?

Method Overloading means **having multiple methods with the same name in the same class, but with different parameters.**

☞ In simple words:

Same method name + Different parameter list = Method Overloading



Simple Real-Life Example

Think of a **calculator**:

- `add(2, 3)`
- `add(2, 3, 4)`
- `add(2.5, 3.6)`

All are **`add( )`**, but work with **different inputs**.



1) Compile-Time Polymorphism (Method Overloading)

Definition:

Same function name but **different parameters**.

Real-Life Example: Mobile Phone 📱

- Call(friend)
- Call(customer care)
- Call(emergency)

☞ Same action **call**, different inputs.

Method Overriding

- **Method Overriding** occurs when a **child class** provides its own **implementation of a method that is already defined in the parent class.**
- ➞ In simple words:
Same method name + Same parameters + Different class = Method Overriding

Real-Life Example

Consider **Bank Interest Rate**:

- RBI → Base rules
- SBI → Own interest calculation
- HDFC → Own interest calculation

All follow **same method name**, but **behavior is different**.

Run-Time Polymorphism (Method Overriding)

Definition:

Child class **redefines** the method of parent class.

Real-Life Example: Vehicle Speed 🚗

- Vehicle $\rightarrow$ speed = 60 km/h
- Car $\rightarrow$ speed = 120 km/h
- Bike $\rightarrow$ speed = 80 km/h

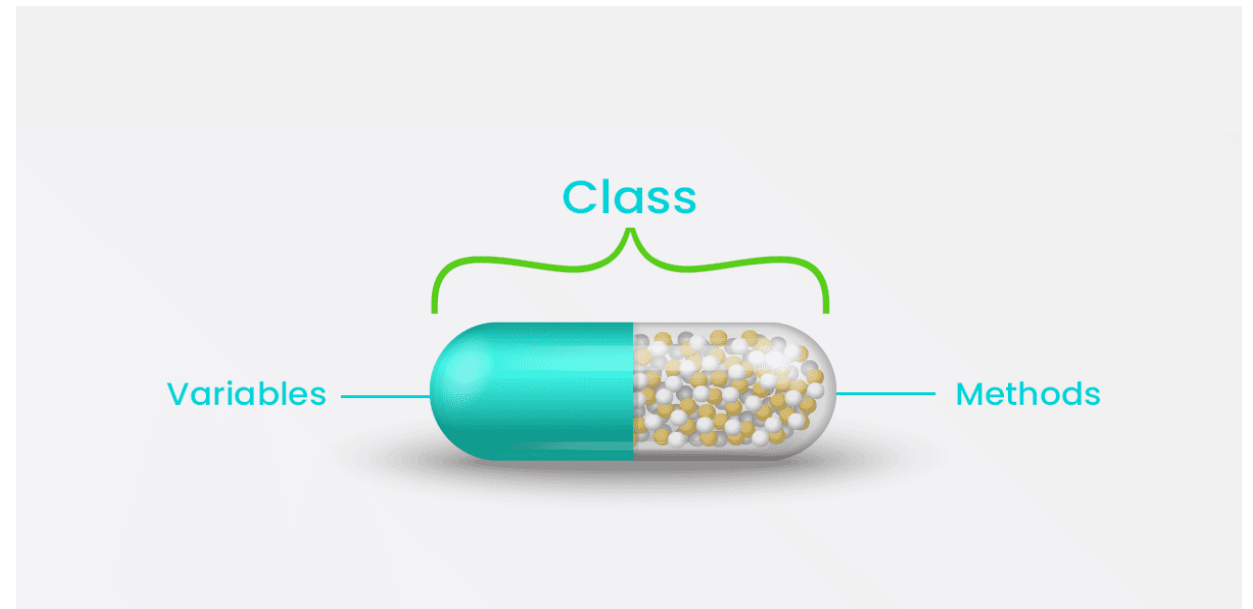
☞ Same method **speed()**, different behavior.



Method Overloading	Method Overriding
Same class	Different class (Inheritance)
Same method name, different parameters	Same method name, same parameters
Compile-time polymorphism	Run-time polymorphism

Abstraction

- Hides **unnecessary details**
- Shows only **essential features**
- Helps focus on **what to do, not how to do**
- Reduces **complexity**



◆ Real-Life Examples

1) ATM Machine

You see:

- Insert card
- Enter PIN
- Withdraw cash

You do not see:

- Bank server processing
- Database operations

👉 Only essential operations are visible → **Abstraction**

2) Car Driving 🚗

You use:

- Steering
- Brake
- Accelerator

You don't know internal engine working.

👉 Interface shown, working hidden → **Abstraction**

Abstraction

Key Concepts of Abstraction:

- Generalization
- Simplification:



1) Generalization

Definition:

Generalization is the process of extracting common features from multiple classes and placing them into a single generalized (parent) class.

 In simple words:

“Find common things → put them in one general class.”

Real-Life Example: Vehicles

Car, Bike, Truck all have:

- start
- stop
- fuel

So we create a **general class** → **Vehicle**

◆ 2) Simplification

Definition:

Simplification is the process of reducing complexity by hiding unnecessary details and keeping only essential features.

👉 In simple words:

“Make complex things simple.”

◆ Real-Life Example: Mobile Phone 📱

You see:

- Call
- Message
- Camera

You don't see:

- Circuit working



Feature	Encapsulation	Data Abstraction
Definition	Wrapping data (variables) and methods (functions) into a single unit (class)	Hiding internal implementation and showing only essential features
Focus	Data protection and binding	Hiding complexity
Purpose	Control access to data	Simplify usage for users
Access	Uses access modifiers (private, public, protected)	Uses abstract classes and interfaces
Level	Implementation level	Design level
Key Idea	“How data is stored and accessed”	“What to show, what to hide”



Interface

- An interface is a collection of abstract methods that define what a class must do, but not how it should do it.
- “Interface shows WHAT to do, not HOW to do.”
- Supports **multiple inheritance**
- Ensures **standard method implementation across classes**

Interface = Rules that classes must follow

Real Life	OOP
Remote buttons	Interface methods
TV	Class implementing interface



Buttons define **what actions can be performed:**

◆ Real-Life Examples

1) Electric Switch & Appliances 🖱️

A switch provides:

- ON
- OFF

You can connect:

- Fan
- Bulb
- AC

👉 Switch = **Interface**

👉 Fan / Bulb / AC = **Implementing classes**

Each device implements the same interface differently.



- **Real-Life Example of Interface**
- **Example: Payment System**
- **Interface: Payment**
- **Method: pay()**
- **Classes implementing it:**
- CreditCardPayment → pay()
- UpiPayment → pay()
- NetBankingPayment → pay()
- **Explanation:**
- All payment methods **must provide pay() functionality**
- Each payment mode has **its own implementation**
- **Real Life Analogy:**
- ATM, UPI, Credit Card → all follow **same payment rules**, but work differently

A **payment interface** may define:



Abstract Class

- An abstract class in OOP (Object-Oriented Programming) is a class that cannot be instantiated (you cannot create its object directly) and is mainly used as a blueprint for other classes.

Abstract class = Incomplete class + Blueprint for child classes



Key Points:

- Cannot create objects directly
- Can have **abstract methods** (no body)
- Can also have **normal methods** (with body)
- Used to achieve **abstraction**

Abstract Class: Vehicle

Common methods:

- start()
- stop()

Abstract method:

- move()

Child classes implement move() differently.

Class	Implementation
Car	drive on road
Bike	ride
Bus	transport passengers

◆ Real-Life Examples

1) Vehicle Example

Vehicle is a general concept:

- Every vehicle must **start()**
- But **how it starts** differs for car, bike, bus

👉 Vehicle = **Abstract class**

👉 Car, Bike, Bus = **Concrete classes**

A **concrete class** is something real and usable,
like:

- Car
- Bike

☞ **Concrete class = Complete class + Can create objects**

2) Shape Example

Shape → area()

But:

- Circle → area formula differs
- Rectangle → area formula differs

 Shape = **Abstract class**

Exception

- An exception is an unexpected error that occurs during program execution and disrupts the normal flow of the program.
- Runtime error that stops normal program flow.

◆ Real-Life Examples

1) ATM Example

- Problem: Insufficient balance
- System shows: **Error message**

👉 This is an exception situation

2) Driving Example

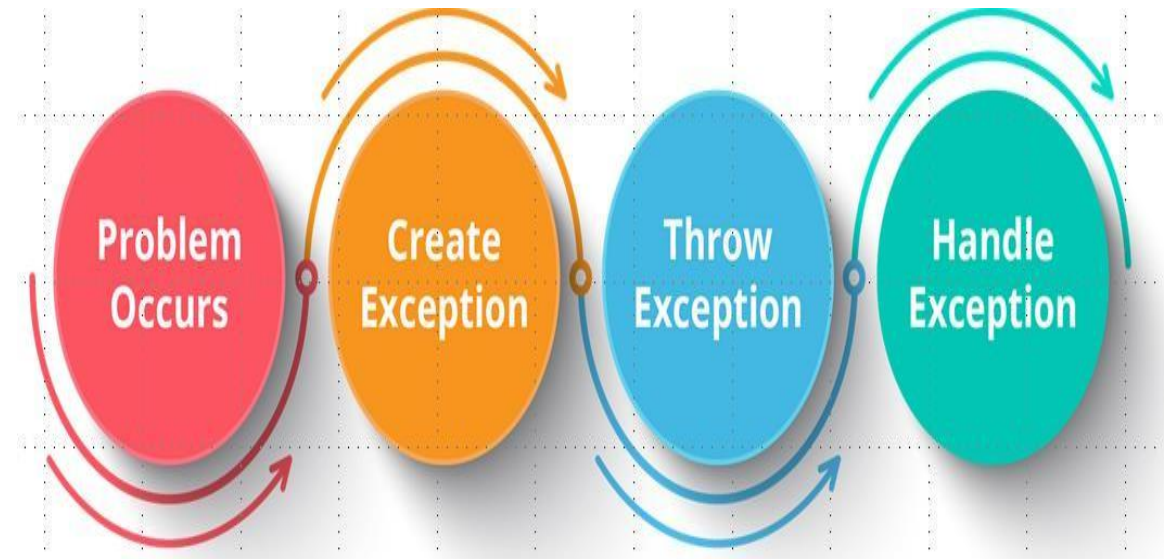
- Normal: Smooth driving
- Exception: Sudden brake failure / accident

👉 Unexpected problem → **Exception**

Exception Handling

Key Concepts of Exception Handling:

- Try-Catch Blocks
- Exception Types
- Exception Propagation
- Exception Handling Mechanisms



Exception Handling

- A programming mechanism to **detect and handle errors**
- Manages **unexpected or exceptional conditions**
- Prevents **abrupt program termination**
- Maintains **normal program flow**

Exceptions include:

- **Runtime errors**
- **Invalid user input**
- **System failures**



Why do we need Exception Handling?

- Prevents **program crash**
- Maintains **normal flow of program**
- Provides **error messages instead of abrupt termination**
- Helps in **debugging**

Example 1: ATM Transaction

•Normal Flow:

- Insert card → Enter PIN → Withdraw cash

•Exception Situation:

- Wrong PIN entered
- Insufficient balance
- Network failure

•Exception Handling:

- Display error message
- Ask user to re-enter PIN
- Cancel transaction safely

Example 2: Online Shopping Payment

- **Normal Flow:**

- Select items → Make payment

- **Exception Situation:**

- Payment failure
 - Network error
 - Invalid card details

- **Exception Handling:**

- Show error message
 - Retry payment
 - Choose another payment method



Exception	Cause
ArithmeticException	Divide by zero
NullPointerException	Access null object
ArrayIndexOutOfBoundsException	Invalid array index
NumberFormatException	Wrong number format
FileNotFoundException	File not found

Different Methods of Exception Handling?

Exception handling can be done using **five main methods / keywords**:

- 1.try – catch**
- 2.try – catch – finally**
- 3.Multiple catch blocks**
- 4.throw keyword**
- 5throws keyword**

◆ 1. try – catch

◆ Real-Life Example: ATM Withdrawal

You go to an ATM and try to withdraw money.

- **try** → You attempt the transaction
- **catch** → If your balance is low, the ATM shows “Insufficient Balance” instead of crashing

👉 **Idea:** Try something risky, and handle the error if it occurs.

◆ 2. try – catch – finally

◆ Real-Life Example: Online Payment

You make an online payment.

- **try** → Payment is processed
- **catch** → If payment fails (network issue), error is shown
- **finally** → Receipt page or session closing happens **no matter what**

👉 **Idea:** Some actions (like closing session or showing confirmation) must always happen.

◆ 3. Multiple catch blocks

◆ Real-Life Example: Login System

When logging in:

- Wrong password → "Incorrect Password"
- Server down → "Server Error"
- Account locked → "Account Blocked"

👉 Different problems are handled in **different ways**.

👉 **Idea:** One try, but **multiple types of error handling**.

◆ 4. throw keyword

◆ Real-Life Example: Age Restriction

At a movie theatre:

- If a person is underage, the system raises an alert immediately and denies entry

👉 Idea: The system itself detects a problem and throws an exception.

◆ 5. throws keyword (Java)

◆ Real-Life Example: Library System

A librarian says:

“This book **may not be available.**”

- The system warns in advance that an issue might occur
- The person using the system must be ready to handle it

👉 **Idea: Declaring beforehand that a function can cause an exception.**

Concept	Real-Life Meaning
try–catch	Try something, handle error if it occurs
try–catch–finally	Handle error + always perform final step
Multiple catch	Handle different errors differently
throw	Manually generate an error
throws	Declare possible error in advance



Constructors: Constructors are special methods that are automatically called whenever an object of a class is created. A constructor is different from normal functions in following ways:

- A constructor has same name as the class itself
- Don't have return type
- Automatically called when an object is created.
- If we do not specify a constructor, C++ compiler generates a default constructor for us (expects no parameters and has an empty body)



```
#include <iostream>

using namespace std;

class A {
public:

    // Constructor without any parameters
    A() {
        cout << "Constructor called" << endl;
    }
};

int main() {
    A obj1;
    return 0;
}
```




Design Patterns

Key Concepts of Design Patterns:

- Problem-Solution Approach
- Abstraction and Encapsulation
- Flexibility and Extensibility
- Common Vocabulary



Categories of Design Patterns:

- Creational Patterns
- Structural Patterns
- Behavioral Patterns

Benefits of Using Design Patterns:

- Code Reusability
- Scalability
- Maintainability
- Performance:



Object-Oriented Analysis and Design (OOAD)

Object-Oriented Analysis and Design (OOAD) is methodology for designing software systems based on object-oriented principles.

Key Concepts of OOAD:

- Object-Oriented Principles
- Requirement Analysis
- Modeling
- Iterative Development
- Design Patterns



Phases of OOAD:

- Requirement Analysis
- System Design
- Object-Oriented Modeling
- Testing
- Deployment



Benefits of OOAD:

- Modularity
- Reusability
- Scalability
- Maintainability
- Quality

◆ What is a Pointer?

A pointer is a variable that stores the address of another variable or object.

👉 Example (C++):

```
</> C++
```

```
int a = 10;
```

```
int *ptr = &a;    // ptr stores address of a
```



Best Practices in OOP

Key Best Practices in OOP:

- Single Responsibility Principle (SRP)
- Open/Closed Principle (OCP)
- Liskov Substitution Principle (LSP)
- Interface Segregation Principle (ISP):



Best Practices in OOP

- Dependency Inversion Principle (DIP)
- Encapsulate What Varies
- Favor Composition Over Inheritance
- Follow Naming Conventions
- Write Clean and Readable Code
- Test-Driven Development (TDD):



Web Links For Reference (Complete Video Course)

- https://youtu.be/cWymldw9VJ0?si=V_6VKJvWf4T17sF2
- <https://youtu.be/vmxTUhn2fBg?si=UhK6GtU4Q7uhHNmn>
- https://youtu.be/Mf2RdpEiXjU?si=_JfDRWoTsoXr-TU2
- <https://youtube.com/playlist?list=PLBlnK6fEyqRiwWLbSXXFtdGV8OVqr9dZr&si=D6Rqtbht4MNHuZqJ>
- <https://youtu.be/mlIUKyZIUUU?si=OpLR2EX9WYF8kOGA>



Web Links For Reference (Study Material)

- https://www.w3schools.com/java/java_oop.asp
- <https://www.geeksforgeeks.org/object-oriented-programming-oops-concept-in-java/>
- https://www.w3schools.com/cpp/cpp_oop.asp
- <https://www.javatpoint.com/java-oops-concepts>
- <https://docs.oracle.com/javase/tutorial/java/concepts/>
- <https://realpython.com/python3-object-oriented-programming/>



NEXT----->: Algorithms and Data Structures

- Algorithms and Data Structures
- Understanding Algorithms and Analysis
- Running Time Analysis and Rate of Growth
- Asymptotic Notation: Big-O Notation
- Data Structures and Recursive Algorithms
- Recursion and Backtracking,, Linked Lists
- Stacks and Queues
- Trees and Binary Search Trees (BSTs)